

# Real-World Context-Engineered Auto-Reply Agent

A confidence-gated LLM support agent for Ramco products, delivered as a Telegram bot

**Author:** Nikhil Kumar Jha

**Platform:** Telegram — @ramco\_assist\_bot

**Hosted:** [caring-gentleness-production-c3e7.up.railway.app](#)

**Models:** Haiku 4.5 (router) · Sonnet 4.6 (drafter)

**Date:** 4 June 2026

**Knowledge:** Ramco products (public)

**Deployment:** Railway (poller + dashboard + Postgres)

**Repo:** [github.com/mrxjha/lumenx-agent](#) · /Real-world-reply-agent

## 1 · OVERVIEW

The agent answers questions about Ramco's enterprise software (ERP, HCM & Global Payroll, Aviation/MRO, Logistics, EAM) on a real messaging platform. An incoming Telegram message is classified by intent, answered from a curated knowledge base with Claude, scored by a small neural network, and then **auto-sent** when confidence is high or routed to a **human-review dashboard** when it is not. Every model call is cost-logged. The objective: automate routine replies while **never hallucinating on pricing or contract terms**, keeping a human in the loop until the model earns trust.

## 2 · SYSTEM ARCHITECTURE



The pipeline is platform-agnostic — the Telegram connector is the only messaging-specific module, so an email/Teams backend could be swapped in without touching the agent logic.

## 3 · BUILD PROCESS

- LLM Wiki.** Five Ramco product lines + a company-policy page rendered to structured Markdown with a cross-reference graph (Karpathy "LLM-wiki" pattern). Pricing is recorded as "*not published publicly*," so correct deflection becomes the grounded answer.
- Cold-start data & Confidence Net.** A bot cannot scrape past chats, so 47 realistic (question → draft → ideal-reply) examples bootstrap the MLP — negatives reproduce feared failures (invented prices, missing citations); positives are grounded and cited.
- Pipeline.** Intent router (cheap model) → context builder → drafter (quality model) with an anti-hallucination prompt → per-call token/cost logging.
- Dashboard.** Streamlit review queue with confidence badge, expandable context window, cost panel, and one-click approve/edit/reject that replies via Telegram.
- Deployment.** One Docker image, three roles (poller / dashboard / web) on Railway with managed Postgres.

## 4 · CONTEXT ENGINEERING

### 4.1 Intent router

A cheap Haiku call labels each message `greeting · pricing · refund · technical · other` and returns JSON with its own confidence. Greetings and off-topic chat are answered politely without invoking the expensive drafter, and the label biases retrieval toward the right wiki pages.

### 4.2 The context window

For each reply the builder assembles four ranked sources into one prompt: (1) the **LLM Wiki** — company policy plus the top pages by keyword overlap with the query, expanded along the cross-reference graph; (2) the **current chat history**; (3) a **summary of past conversations**; and (4) a **feedback log** of previously approved Q→A pairs for the same intent. The dashboard can show this exact window per reply for auditability.

### 4.3 Anti-hallucination guardrail

The drafter's system prompt forbids inventing prices, refund windows, or contract terms; if the answer is not in the loaded wiki it must say so and offer to route to sales. Facts are cited inline with source tags (e.g. `[ramco_hcm]`, `[company_policy]`), which also feed the confidence model.

### 4.4 Confidence Net (the routing brain)

A small scikit-learn MLP (hidden layers 32→16) scores each draft 0–1 — "would a human send this as-is?" — using only signals available *before* sending: draft length/word-count, digit count (pricing-risk), citation count, hedge-phrase presence, sign-off compliance, retrieval-hit count, and the intent one-hot. Labels come from edit-distance between the draft and the human-sent reply (near-duplicate → 1; heavily edited/rejected → 0) plus explicit thumbs feedback. Drafts scoring  $\geq$  a configurable threshold (default **0.90**) qualify for auto-send; a global `AUTO_SEND_ENABLED` switch keeps everything in human review during cold-start.

## 5 · DEPLOYMENT

One Docker image is dispatched into three roles via a `SERVICE_ROLE` variable on Railway, all sharing a managed Postgres database:

| Service          | Role                   | Public?              | Function                                                 |
|------------------|------------------------|----------------------|----------------------------------------------------------|
| <b>poller</b>    | <code>poller</code>    | No                   | Telegram <code>getUpdates</code> loop → pipeline → route |
| <b>dashboard</b> | <code>dashboard</code> | Yes — the hosted URL | Human review queue + cost dashboard                      |
| Postgres         | —                      | No                   | Shared state: threads, drafts, feedback, token usage     |

*Engineering notes: the corporate network blocks Telegram, so the live bot is driven from a mobile device while the build/test loop runs locally against the (reachable) model API — Railway runs outside that firewall. Deploy fixes: a Streamlit health-check path (so traffic isn't routed before the app binds), a Postgres connect-timeout, and suppression of HTTP-client logs that would otherwise print the bot token.*

6 · PERFORMANCE ANALYSIS

Measured on a balanced 10-query set (pricing, refund, technical, other, greeting) run through the full production pipeline against the live models.

100%

INTENT ACCURACY

4.3 s

AVG REPLY LATENCY

\$0.0099

COST / REPLY

0%

HALLUCINATION RATE

6.1 Accuracy & 6.2 Speed

The intent router labelled **10 / 10** queries correctly; on grounded drafts the Confidence Net scored 0.99–1.00. End-to-end latency (routing + retrieval + drafting):

| Latency   | Average | Median | Min    | Max    |
|-----------|---------|--------|--------|--------|
| Per reply | 4.30 s  | 4.34 s | 3.47 s | 5.25 s |

Drafting (Sonnet) dominates latency; routing (Haiku) is sub-second. A "typing..." indicator is shown while drafting.

6.3 Cost

| Component          | Model             | Avg / reply | Share |
|--------------------|-------------------|-------------|-------|
| Intent routing     | Claude Haiku 4.5  | \$0.00058   | 6%    |
| Reply drafting     | Claude Sonnet 4.6 | \$0.00928   | 94%   |
| Confidence scoring | local MLP         | \$0.00000   | 0%    |
| Total              |                   | \$0.00986   | 100%  |

≈ **\$0.99 per 100 replies** (≈ \$9.86 / 1,000). The cheap-router / quality-drafter split keeps classification effectively free; local MLP scoring adds zero marginal cost.

6.4 Hallucination rate

Across the 4 sensitive (pricing/refund) queries the agent quoted **0 fabricated prices or terms** — all four deflected to a custom quote / the signed contract, with citations. An automated keyword check flagged 1 reply, but manual review showed a **false positive**: it described the pricing *model* ("per employee, per country") while stating there is no published fee — no number was invented. **True hallucination rate: 0%.**

6.5 Confidence model

Bootstrapped on **47** generated examples (29 positive / 18 negative). It fits this small distribution cleanly; a held-out split and weekly retraining on real dashboard feedback are the path to a calibrated threshold.

7 · LIMITATIONS & NEXT STEPS

- The MLP is bootstrapped on synthetic data; real dashboard approve/edit/reject feedback will replace it and enable a proper held-out evaluation and threshold calibration.
- Wiki relevance uses keyword overlap; embeddings would improve retrieval on paraphrased questions.
- Hallucination is measured by heuristic + manual review on a small set; an LLM-graded rubric on a larger held-out set would tighten the estimate. Prompt caching on the reused wiki block would cut drafting cost further.

Figure — paste screenshots here: (1) dashboard review queue showing a confidence score + the expandable context window, and (2) the Telegram chat showing a pricing deflection.